What is a SQL Constraint?

A **SQL constraint** is a **rule applied to a table column or table** that controls what data is allowed.

Constraints protect data quality and enforce business rules.

They are defined when creating a table or added later.

Simple Definition

Constraint means:

“Only valid data can enter the table.”

Why We Need Constraints

Without constraints, users/apps can insert bad data like:

- duplicate employee IDs
- negative salary
- null email
- invalid department reference
- duplicate usernames

Constraints stop these problems **inside the database itself**.

Real Meaning

Even if application code has bugs, constraints still protect the data.

That is why serious systems rely on them.

Example Table Without Constraints

employees

id name salary

1 Ravi 50000

1 Indra -2000

NULL Ajay 30000

Problems:

- duplicate id
- negative salary

- null id

Bad data entered.

Same Table With Constraints

```
CREATE TABLE employees (
  id INT PRIMARY KEY,
  name VARCHAR(50) NOT NULL,
  salary DECIMAL(10,2) CHECK (salary >= 0)
);
```

Now invalid rows are rejected.

Common Types of Constraints

Constraint	Purpose
NOT NULL	Value must be present
UNIQUE	No duplicate values
PRIMARY KEY	Unique + not null row identifier
FOREIGN KEY	Must match parent table value
CHECK	Must satisfy condition
DEFAULT	Gives automatic default value

---

1. NOT NULL

Meaning

Column cannot be empty.

```
name VARCHAR(50) NOT NULL
```

Now:

```
INSERT INTO employees(name) VALUES(NULL);
```

Rejected.

Why Needed

Some fields are mandatory:

- name

- email
- order_date

---

2. UNIQUE

Meaning

No duplicates allowed.

email VARCHAR(100) UNIQUE

Only one user per email.

Why Needed

Useful for:

- username
- email
- Aadhaar-like IDs
- phone numbers (sometimes)

---

3. PRIMARY KEY

Meaning

Uniquely identifies each row.

id INT PRIMARY KEY

Rules:

- unique
- not null

Why Needed

Every row should be identifiable.

Example:

Employee ID, Order ID.

---

4. FOREIGN KEY

Meaning

Child table value must exist in parent table.

Example:

Departments table:

dept_id	dept_name
10	HR
20	IT

Employees:

emp_id	dept_id
1	10

Constraint:

FOREIGN KEY (dept_id) REFERENCES departments(dept_id)

Now dept_id = 99 rejected if not present.

Why Needed

Keeps relationships valid.

---

5. CHECK

Meaning

Custom condition.

CHECK (salary >= 0)

Rejects negative salary.

Other examples:

- age >= 18
- quantity > 0
- gender IN (...)

---

6. DEFAULT

Meaning

Auto value if not given.

status VARCHAR(20) DEFAULT 'ACTIVE'

If omitted, database inserts ACTIVE.

Deep Internal Purpose of Constraints

Constraints are part of **data integrity**.

Types of Integrity

Type	Example
Entity Integrity	Primary key unique, not null
Domain Integrity	Correct allowed values
Referential Integrity	Foreign keys valid
Business Integrity	Salary > 0

Why Constraints Better Than App Validation Alone

Suppose:

- Web app checks salary > 0
- Another script inserts directly into DB and skips checks

Without DB constraints:

Bad data enters.

With constraints:

Database rejects from any source.

That's why constraints are powerful.

What Happens Internally

When INSERT/UPDATE runs:

INSERT INTO employees VALUES (1,'Indra',-5000);

Database checks constraints before commit.

If violated:

- query fails
- row not inserted

Performance Note

Constraints add some checking cost, but benefits are worth it because:

- clean data
- fewer bugs

- trustable analytics
- safe relationships

Real Company Examples

E-commerce

- order_id primary key
- quantity check > 0
- customer_id foreign key

Banking

- account_no unique
- balance rules
- customer exists

HR

- emp_id primary key
- email unique
- dept valid

Bad Design Without Constraints

If no constraints:

- duplicate customers
- orphan orders
- invalid salaries
- null important fields

Then reporting becomes unreliable.

Final Summary Table

Constraint	Why Needed
NOT NULL	Mandatory fields
UNIQUE	Prevent duplicates
PRIMARY KEY	Identify rows

Constraint	Why Needed
FOREIGN KEY	Valid relationships
CHECK	Enforce rules
DEFAULT	Auto values

One-Line Final Answer

SQL constraints are database rules that ensure only valid, consistent, and meaningful data is stored.

"why constraints matter?"

They enforce data integrity at the database level, regardless of which application inserts the data.

Why This Error Happened

Error Code: 3819. Check constraint 'orders_chk_1' is violated

Means:

You are trying to add a **CHECK constraint**, but some **existing rows in orders table already contain invalid order_status values**.

Your Command

```
ALTER TABLE orders
ADD CHECK (
 order_status IN ('CLOSED','COMPLETE','PROCESSED','PENDING')
);
```

This tells MySQL:

Only these 4 values are allowed.

But MySQL first checks all current rows.

If even one row has:

CANCELLED

NULL

done

closed (wrong case maybe depending rules)

SHIPPED

Then constraint creation fails.

Why Database Does This

Because if invalid rows already exist, adding the rule would make table inconsistent immediately.

So MySQL blocks it.

How to Find Bad Rows

Run:

```
SELECT DISTINCT order_status
FROM orders
WHERE order_status NOT IN
('CLOSED','COMPLETE','PROCESSED','PENDING')
  OR order_status IS NULL;
```

This shows invalid values.

How to Fix

Option 1: Update Bad Values

Example:

```
UPDATE orders
SET order_status = 'PENDING'
WHERE order_status = 'WAITING';
```

Then add constraint again.

Option 2: Delete Bad Rows (if unwanted)

```
DELETE FROM orders
WHERE order_status NOT IN
('CLOSED','COMPLETE','PROCESSED','PENDING');
```

Use carefully.

Option 3: Include Existing Values in Constraint

If CANCELLED is valid business status:

```
ALTER TABLE orders
ADD CHECK (
 order_status IN
('CLOSED','COMPLETE','PROCESSED','PENDING','CANCELLED')
);
```

Best Practice Before Adding CHECK

Always inspect data first:

```
SELECT order_status, COUNT(*)
FROM orders
GROUP BY order_status;
```

Simple Meaning

Your table already has statuses outside allowed list.

One-Line Final Answer

MySQL refused the CHECK constraint because existing rows in orders.order_status violate the allowed values list.

Honest Advice

In real projects, first clean old data, then add constraints.
That prevents future bad inserts.

If you share the output of:

```
SELECT DISTINCT order_status FROM orders;
```

I can tell you exactly which rows are causing the error.

DROP CONSTRAINT & ADD CONSTRAINT in SQL

Explained Simply + Many Examples + Real-Time Scenarios

Constraints are rules on table columns.

Examples:

- no duplicate email
- salary must be positive
- customer must exist
- status only certain values

Sometimes business rules change. Then we **add**, **remove**, or **modify** constraints.

---

1. What is ADD CONSTRAINT?

Used to apply a new rule to an existing table.

"From now on, follow this rule."

Syntax

ALTER TABLE table_name

ADD CONSTRAINT constraint_name constraint_type (...);

---

2. What is DROP CONSTRAINT?

Used to remove an existing rule.

"This rule is no longer needed."

Syntax (varies by DB)

ALTER TABLE table_name

DROP CONSTRAINT constraint_name;

MySQL sometimes uses:

ALTER TABLE table_name DROP INDEX index_name;

ALTER TABLE table_name DROP FOREIGN KEY fk_name;

Real Example Table

employees

id | name | email | salary | dept_id

ADD CONSTRAINT Examples

Example 1: Add UNIQUE on Email

```
ALTER TABLE employees
ADD CONSTRAINT uq_email UNIQUE(email);
```

Meaning

No two employees can have same email.

Real Scenario

Company employees login using email.

Duplicate emails create chaos.

Example 2: Add CHECK on Salary

```
ALTER TABLE employees
ADD CONSTRAINT chk_salary
CHECK (salary >= 0);
```

Meaning

Negative salary not allowed.

Real Scenario

Payroll mistake prevention.

Example 3: Add PRIMARY KEY

```
ALTER TABLE employees
ADD CONSTRAINT pk_emp PRIMARY KEY(id);
```

Meaning

Each employee must have unique ID.

Example 4: Add FOREIGN KEY

```
ALTER TABLE employees
ADD CONSTRAINT fk_dept
FOREIGN KEY (dept_id)
REFERENCES departments(dept_id);
```

Meaning

Employee department must exist.

Real Scenario

No employee assigned to fake department.

Example 5: Add DEFAULT

```
ALTER TABLE orders
ALTER status SET DEFAULT 'PENDING';
```

(Depends on database)

Meaning

If no status given, set PENDING.

DROP CONSTRAINT Examples

Example 1: Drop UNIQUE Constraint

```
ALTER TABLE employees
DROP CONSTRAINT uq_email;
```

Why?

Business now allows shared team email addresses.

Example 2: Drop CHECK Constraint

```
ALTER TABLE employees
DROP CONSTRAINT chk_salary;
```

Why?

Need to allow negative adjustments temporarily (recovery entries).

Example 3: Drop FOREIGN KEY

```
ALTER TABLE employees
DROP CONSTRAINT fk_dept;
```

Why?

Migrating systems and relationships changing.

Example 4: Drop Primary Key

```
ALTER TABLE employees
DROP PRIMARY KEY;
```

(MySQL style)

Why?

Changing ID system to composite key or UUID.

Real-Time Scenarios (Important)

Scenario 1: Startup Growth

Initially:

email UNIQUE

Later company allows multiple accounts under same email.

So:

```
DROP CONSTRAINT uq_email;
```

Scenario 2: New Business Rule

Earlier any order_status allowed.

Now strict statuses only:

```
ADD CHECK (
status IN ('PENDING','SHIPPED','DELIVERED')
);
```

Scenario 3: ERP Migration

Old department IDs inconsistent.

Drop FK temporarily:

DROP CONSTRAINT fk_dept;

Load corrected data.

Then add FK again.

Scenario 4: Banking Safety

No overdraft allowed:

ADD CHECK (balance >= 0);

Important Warning Before ADD CONSTRAINT

If old data violates rule, command fails.

Example:

ADD CHECK (salary >=0)

But table has:

salary = -5000

Then error occurs.

Always Check Existing Data First

SELECT * FROM employees WHERE salary < 0;

Important Warning Before DROP CONSTRAINT

Dropping rules can allow bad data later.

Example:

Drop UNIQUE(email)

Then duplicates may enter.

Use carefully.

How to See Constraint Names

SHOW CREATE TABLE employees;

or

DESC employees;

Best Practice in Real Companies

Before Adding Constraint:

1. Analyze current data
2. Clean invalid rows
3. Add rule

Before Dropping Constraint:

1. Understand why it exists
2. Check application dependency
3. Backup if needed

Quick Summary Table

Action	Meaning
ADD CONSTRAINT	Add new rule
DROP CONSTRAINT	Remove old rule
CHECK	Validate values
UNIQUE	No duplicates
FK	Valid relationship
PK	Unique row ID

Interview Answer

Why use ADD CONSTRAINT?

To enforce new data quality/business rules.

Why use DROP CONSTRAINT?

To remove outdated rules or during migrations.

Final One-Line Answer

ADD CONSTRAINT adds data rules to a table, while DROP CONSTRAINT removes rules when business needs change.

Honest Advice

In real data engineering work, constraints save you from dirty data. Learn them deeply—they matter more than many beginners think.